

# METAHEURÍSTICAS DE BUSCA LOCAL - CONTINUAÇÃO

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 22 de setembro de 2025

Iago Carvalho

Departamento de Ciência da Computação



# INTENSIFICAÇÃO E ESTAGNAÇÃO

Nas aulas anteriores vimos algoritmos de busca local simples e meta-heurísticas um pouco mais complexas

Algoritmos como *Hill Climbing* e o *Multi-Start* não conseguem fugir de ótimos locais

- São mecanismos de intensificação
- Costumam estagnar (ficar presos em ótimos locais) rápido

Meta-heurísticas, como o VND, VNS e GRASP são mais capazes de escapar destes ótimos locais

Hoje veremos mais duas meta-heurísticas capazes de escapar de ótimos locais

- Iterated Local Search (ILS)
- Tabu Search (TS)

# ITERATED LOCAL SEARCH

# ITERATED LOCAL SEARCH

A meta-heurística ILS foi proposta por Lourenço, Olivier e Stützle

- *Lourenço, Helena R., Olivier C. Martin, and Thomas Stützle. "Iterated local search." Handbook of metaheuristics. Boston, MA: Springer US, 2003. 320-353* [▶ Link](#)
- Uma revisão recente sobre o tema pode ser encontrada em *Ramalhinho, Helena, and Thomas Stützle. "Iterated local search, iterated greedy and applications." Top 33.2 (2025): 229-261* [▶ Link](#)

O ILS une características do *Multi-Start* e do VNS em um único algoritmo

- Múltiplos reinícios do *Multi-Start*
- *Shake* ou *perturbação* do VNS

# ITERATED LOCAL SEARCH

O ILS utiliza uma heurística construtiva uma única vez

Logo após, ele realiza uma busca local na solução inicial

Finalmente, é dado início ao processo de busca do ILS

1. *Shake*
2. Busca local

O ILS é baseado na hipótese de que ótimos locais de alta qualidade tendem a estar agrupados em certas regiões do espaço de busca

- Ao perturbar um bom ótimo local, há uma chance maior de encontrar outro ótimo local ainda melhor nas proximidades do que começando de um ponto completamente aleatório

---

## Algoritmo 1: Iterated Local Search

---

**Entrada:**  $N_k(s)$  // Estruturas de vizinhança

**Entrada:**  $s$  // Solução inicial

1  $s \leftarrow \text{buscaLocal}(s, N(s))$  // Realiza busca local

2 **enquanto** critério de parada não atingido **faça**

3      $s' \leftarrow \text{shake}(s, N_k(S))$

4      $s'' \leftarrow \text{buscaLocal}(s', N_k(s'))$

5      $s \leftarrow \text{melhor}(s, s'')$  // Salva a melhor solução

6 **retorne**  $s$

---

# ITERATED LOCAL SEARCH

Assim como no GRASP e no VNS, podemos utilizar qualquer mecanismo de busca local

- Hill Climbing
- VND
- ...

Devemos ter um *shake* de força controlada

- Se for muito fraco, a busca local retornará para o mesmo ótimo local
- Se for muito forte, destruiremos a busca atual
  - O algoritmo será semelhante ao *Multi-Start*

Também é possível construir heurísticas ILS adaptativas

- Adaptar a força do *shake* de forma semelhante ao realizado com o VNS

# TABU SEARCH



A Tabu Search (TS) foi proposta em dois artigos consecutivos

1. Glover, Fred. "Tabu search—part I." ORSA Journal on computing 1.3 (1989): 190-206 [▶ Link](#)
2. Glover, Fred. "Tabu search—part II." ORSA Journal on computing 2.1 (1990): 4-32 [▶ Link](#)

A TS é muito diferente de tudo o que vimos até agora

- Ela utiliza o conceito de **memória de curto prazo** (lista tabu)
- Além disso, ela também permite movimentos para soluções de **pior qualidade**

# TABU SEARCH - LISTA TABU

A lista tabu é uma **estrutura de dados** que armazena atributos de movimentos recentes

- Por exemplo, se uma vizinhança *swap* de um TSP trocou a cidade  $i$  de lugar com a  $j$ , a lista tabu vai armazenar a tupla  $\langle i, j \rangle$

Atributos salvos na lista tabu não podem ser realizados novamente

- Em outras palavras, qualquer algoritmo de busca que envolva os itens da busca tabu é proibido

Lista tabu tem um **tamanho** (ou duração) máxima

- Se muito pequena, é inefetiva
- Se muito grande, ela *sufoca* a busca
- Essa duração pode ser fixa ou dinâmica

# TABU SEARCH - CRITÉRIO DE ASPIRAÇÃO

É um mecanismo que permite revogar o status tabu de um movimento sob certas condições

- O critério mais comum é se o movimento tabu levar a uma solução aprimorante *global*
  - Melhor solução encontrada em toda a busca

O critério de aspiração garante que os movimentos proibidos na lista tabu não impeçam o algoritmo de alcançar um novo ótimo global

---

## Algoritmo 2: Tabu Search

---

**Entrada:**  $N_k(s)$  // Estruturas de vizinhança  
**Entrada:**  $s$  // Solução inicial  
**Entrada:**  $T \leftarrow \emptyset$  // Lista tabu vazia

- 1  $s^* \leftarrow s$  //  $s^*$  é a melhor solução global
- 2 **enquanto** critério de parada não atendido **faça**
- 3      $s' \leftarrow \text{buscaLocal}(s, N_k(s), T)$  // Admite piora
- 4      $\text{atualiza}(T)$  // Insere e remove atributos
- 5      $s \leftarrow s'$  // Atualiza a solução corrente
- 6      $s^* \leftarrow \text{melhor}(s', s^*)$  // Atualiza a melhor solução
- 7 **retorne**  $s^*$

---

# TABU SEARCH - TAMANHO DA LISTA

O tamanho da lista é igual a um certo número de iterações do algoritmo

- Quanto maior a lista, mais restritivos são os movimentos
  - Foco em exploração
- Quanto menor a lista, mais próximo é a busca tabu de uma busca local simples
  - Foco em intensificação

Listas muito grandes aumentam o tempo de execução do algoritmo e podem limitar seu desempenho

- É preciso verificar muitas exceções
- Algoritmo tem pouca margem de movimento

# COMPARAÇÃO ENTRE ALGORITMOS DE BUSCA LOCAL

**VNS-VND:** Exploração sistemática de múltiplas estruturas de vizinhança; sistema de *shake* para escapar de ótimos locais

**GRASP:** Múltiplos inícios seguidos de busca local

**ILS:** Realiza um *passeio* entre ótimos locais, combinando o *shake* do VNS e o esquema de reinício de outros algoritmos

**TS:** Uso de memória adaptativa para proibir movimentos recentes e evitar ciclagem

**VNS-VND:** Exploração de múltiplas vizinhanças; Trajetória única com escape sistemático e estocástico

**GRASP:** Múltiplas trajetórias independentes (esquema de *Multi-start*)

**ILS:** Trajetória única com escape estocástico

**TS:** Trajetória única com escape guiado por memória



**VNS-VND:** Algoritmo de *shake*; utilização de múltiplas vizinhanças

**GRASP:** Reinício com iterações independentes

**ILS:** Algoritmo de *shake*

**TS:** Uso de memória (lista tabu)

# PARÂMETROS CRÍTICOS

**VNS-VND:** Conjunto e ordem das estruturas de vizinhança  $N_k(s)$ ;  
Maneira como as vizinhanças são trocadas; Força do *shake*

**GRASP:** O valor de  $\alpha$

**ILS:** Força do *shake*

**TS:** Tamanho (ou duração) da lista tabu; Critério de Aspiração

## Parâmetro comum

Além disso, todas as heurísticas tem como parâmetro crítico a definição de seus critérios de parada

## PONTOS FORTES

**VNS-VND:** Simples e poderoso devido a exploração de múltiplas vizinhanças e a utilização de *shake*; Bom equilíbrio entre exploração e intensificação

**GRASP:** Fácil implementação, inclusive de forma paralela; Múltiplas trajetórias são boas para exploração

**ILS:** Capaz de gerar múltiplos ótimos locais; Eficaz ao focar em regiões de alta qualidade

**TS:** Uso de memória previne ciclos; Geralmente é a melhor heurística de busca local para um problema

**VNS-VND:** Alto custo computacional; Sua performance é dependente das estruturas de vizinhança escolhidas; Necessária a definição de múltiplos esquemas de vizinhança diferentes

**GRASP:** Falta de aprendizado ao longo da busca devido a independência das iterações; Altamente dependente da heurística construtiva

**ILS:** Difícil de parametrizar o mecanismo de shake (muito fraco, muito forte)

**TS:** Complexo de implementar e de parametrizar (tamanho da lista tabu)